Librería Log Transaccional

- Introducción
- Características
- Prerequisitos
- Dependencias
- Configuración
- Implementación en Microservicios

Introducción

La librería es un artefacto utilitario que permite crear el mensaje del log transaccional y colocarlo en el tópico de kafka.

Características

- Librería agnóstica a Spring boot 3.5.12.
- Compilada en versión Java 21.

Implementación

Para implementar la librería en un microservicio, se debe realizar las siguientes configuraciones:

1. Agregar las siguientes dependencias

Mvc

Copiar la siguiente dependencia según el tipo de compilación:

Gradle

```
implementation 'com.pichincha.co
```

Maven

```
<dependency>
  <groupId>com.pichincha.common<
  <artifactId>lib-event-logs-mvc
  <version>1.0.0</version>
</dependency>
```

WebFlux

Copiar la siguiente dependencia según el tipo de compilación:

Gradle

```
implementation 'com.pichincha.co
```

Maven

```
<dependency>
  <groupId>com.pichincha.common<
  <artifactId>lib-event-logs-web
  <version>1.0.0</version>
</dependency>
```

2. Sincronizar el archivo **build.gradle**

Configuración

1. Configuración del archivo application.yml

> Importante
>
> Se debe cambiar los parámetros correspondientes a servidor y tópico.

```
spring:
  kafka:
    security:
      protocol: PLAINTEXT
    properties:
      sasl:
        mechanism: PLAIN
        jaas:
          config: ""
      bootstrap:
        servers: ${KAFKA_SERVER}
      session:
        timeout:
          ms: 45000
      request:
        timeout:
          ms: 2000
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSeria
      value-serializer: org.apache.kafka.common.serialization.StringSer

logging:
  level:
    org:
      apache:
        kafka: OFF
  event:
    mode: 'EXTERNAL'
    kafka:
      topic:
        name: ${KAFKA_TOPIC_AUDITOR}
  excluded-paths: /actuator/**,/health,/metrics,/prometheus
    executor:
      isDefault: false
      corePoolSize: ${THREAD_CORE_POOL_SIZE}
      maxPoolSize: ${THREAD_MAX_POOL_SIZE}
      keepAliveTime: ${THREAD_KEEP_ALIVE_TIME}
      queueSize: ${THREAD_QUEUE_SIZE}
xml:
  template:
    templates:
      "TipoTransaccion": ${XML_TRANSACCION_0001}
```

> Recuerde que el parámetro: `logging.level.org.apache.kafka=OFF` apaga los logs en el pod que hacen referencia a kafka.

2. Atributos:

2.1 Kafka:

Campo Variable de ambiente	Requerido	Tipo	Default	Descripción
`spring.kafka.properties.sasl.mechanism`	**SI**	***String***	`PLAIN`	Mecanismo SASL para Kafka
`spring.kafka.properties.bootstrap.servers`	**SI**	***String***	`${KAFKA_SERVER}`	Servidores de inicio de Kafka
`spring.kafka.properties.security.protocol`	**SI**	***String***	`SASL_SSL`	Protocolo de seguridad de Kafka
`spring.kafka.properties.session.timeout.ms`	**SI**	***Number***	`45000`	Tiempo de espera de sesión para Kafka
`spring.kafka.properties.request.timeout.ms`	**SI**	***Number***	`2000`	Tiempo de espera para una Solicitud a Kafka
`spring.kafka.producer.key-serializer`	**SI**	***String***	`org.apache.kafka.common.serialization.StringSerializer`	Clase serializadora de claves para el productor de Kafka

`spring.kafka.producer.value-serializer`	**SI**	***String***	`org.apache.kafka.common.serialization.StringSerializer`	Clase de serializador de valor para el productor de Kafka

2.2 Logging:

Campo Variable de ambiente	Requerido	Tipo	Default	Descripción
`logging.event.mode`	**SI**	***String***	`EXTERNAL`	Modo para el registro del auditor
`logging.event.kafka.topic.name`	**SI**	***String***	`${KAFKA_TOPIC_AUDITOR}`	Tópico de Kafka para registrar eventos de auditoría
`logging.event.executor.isDefault`	**SI**	***Boolean***	`false`	Bandera para indicar si el ejecutor debe utilizar la configuración predeterminada
`logging.event.executor.corePoolSize`	**SI**	***Number***	`${THREAD_CORE_POOL_SIZE}`	Tamaño del grupo de núcleos para el ejecutor del auditor de registro

`logging.event.executor.maxPoolSize`	**SI**	***Number***	`${THREAD_MAX_POOL_SIZE}`	Tamaño máximo del grupo para el ejecutor del auditor de registro
`logging.event.executor.keepAliveTime`	**SI**	***Number***	`${THREAD_KEEP_ALIVE_TIME}`	Tiempo de vida para el ejecutor
`logging.event.executor.queueSize`	**SI**	***Number***	`${THREAD_QUEUE_SIZE}`	Tamaño de la cola para el ejecutor
`logging.event.excluded-paths`	**SI**	**String**	`${EXCLUDE_PATH}`	Paths para se excluyen de la librería

Implementación en Microservicios

1. Detalle de los componentes de la librería:

Nombre	Descripción	Parametros
`@EventAudit`	Es una anotación personalizada que se utiliza en el adaptador para la captura del request y response del api/microservicio invocado para enviar el mensaje al tópico.	**Service**: Colocar el nombre del servicio a invocar. **Method**: Colocer el nombre del metodo del servicio a invocar **Type**: AuditType.T , se refiere que es la auditoria de tipo transaccional.

2. **Implementación**

Incluir las plantillas en el archivo **application.yml**

Cápsulas de ayuda: Estas plantillas deben ser colocados en el helm como variables de entorno.

- **Plantillas :** [Plantillas xml -shared]

Ejemplo:

```
xml:
  template:
    templates:
      "201000101": |
        <PLANTILLA servicio="WSClientes0001" metodo="ConsultarCuentasAc
         <TX cargaFuente="*" />
         <RX cargaFuente="cuentas">
          <coleccion nombrePadre="corrientes" nombreHijo="corriente" fu
           <campo nombre="numeroCuenta" fuente="/" nomenclatura="numero
           <campo nombre="tipo" fuente="/" nomenclatura="tipo" />
          </coleccion>
          <coleccion nombrePadre="ahorros" nombreHijo="ahorro" fuentePa
           <campo nombre="numeroCuenta" fuente="/" nomenclatura="numero
           <campo nombre="tipo" fuente="/" nomenclatura="tipo" />
          </coleccion>
          <coleccion nombrePadre="inversiones" nombreHijo="inversion" f
           <campo nombre="numeroOperacion" fuente="/" nomenclatura="num
           <campo nombre="tipo" fuente="/" nomenclatura="tipo" />
          </coleccion>
         </RX>
        </PLANTILLA>
```

WebFlux

1. Añadir la dependencia al proyecto.

Colocar la anotación `@EventAudit` *al adaptador para proyectos con webflux, utilizar WebClient.*

Ejemplo consumo uso

```
@Component
@RequiredArgsConstructor
public class WSClientes0001Adapter implements Clientes0001Port {

    private final WebClient wsclientes0001WebClient;
    private static final String ERROR_MESSAGE = "Error consultando cuen
    private static final String ERROR_CODE_OK = "0";

    @Override
    @EventAudit(service = "WSClientes0001", method = "ConsultarCuentasA
    public Mono<ConsultarCuentasActivas01ResponseDto> consultarCuentasA
        log.info("Invocando WSClientes0001 - ConsultarCuentasActivas01"
        return executeRequest(requestDto)
                .doOnNext(responseDto ->
                        log.info("Respuesta recibida de WSClientes0001:
                .onErrorMap(this::mapError)
                .flatMap(this::validateResponse);
}
```

MVC

1. Añadir la dependencia al proyecto.

> *Colocar la anotación* `@EventAudit` *al adaptador para proyectos mvc/soap, utilizar RestClient.*

```
@Component
@RequiredArgsConstructor
public class WSClientes0001RestAdapter implements Clientes0001Port {

    protected final RestClientFactory restClientFactory;
    protected RestClient restClient;
    @Value("${api.clientes0001.uri}")
    private String baseUrl;
    private static final String ERROR_CODE_OK = "0";

    @Override
    @EventAudit(service = "WSClientes0001", method = "ConsultarCuentasA
    public ConsultarCuentasActivas01ResponseDto consultarCuentasActivas
        log.info("Invocando WSClientes0001 - ConsultarCuentasActivas01"
        // 1. Construir request DTO
        ConsultarCuentasActivas01RequestDto request = buildConsultarCue
        // 2. Ejecutar request con XML raw
        ConsultarCuentasActivas01ResponseDto response = post(request, C
        log.info("Respuesta recibida de WSClientes0001: {}", response);
        // 3. Validar respuesta de negocio (lanza ApiBusinessException
        return validateResponse(response);
    }

    protected <T, R> R post(T request, Class<R> responseType) {

        try {
            R response = restClient
                    .post()
                    .uri(baseUrl)
                    .body(request)
                    .retrieve()
                    .onStatus(HttpStatusCode::is4xxClientError, (req, r
                        String errorMsg = String.format("Error 4xx en A
                        log.error("❌ {}", errorMsg);
                        throw new ApiClientErrorException(errorMsg, bas
                    })
                    .onStatus(HttpStatusCode::is5xxServerError, (req, r
                        String errorMsg = String.format("Error 5xx en A
                        log.error("❌ {}", errorMsg);
                        throw new ApiServerErrorException(errorMsg, bas
                    })
                    .body(responseType);
            return response;
        } catch (ApiClientErrorException | ApiServerErrorException e) {
            throw e;
        } catch (Exception e) {
            log.error("❌ Error inesperado en POST: {}", e.getMessage()
            throw new RestClientException("Error invocando API: " + bas
        }
    }
}
```

3. **Mensaje tópico**

```
{
  "logs": {
```

```
    "empresa": "0010",
    "canal": "10",
    "medio": "100001",
    "aplicacion": "00738",
    "agencia": "12",
    "idioma": "ec-ES",
    "usuario": "USINTERT",
    "serverConsumer": "0.0.0.0",
    "sesion": "0057b5a2-21da-4997-885f-0fe82b69f3b4",
    "unicidad": "1",
    "guid": "c5738900367248a9bcff168cd11007c1",
    "fechaHora": "202409031659150372",
    "detalleCliente": {
      "ip": "0.0.0.0",
      "dispositivo": "4f983255-d449-4488-9",
      "geolocalizacion": "",
      "idCliente": "18092025",
      "tipoIdCliente": "10"
    },
    "detalleOrquestador": {
      "nombreServicio": "mdw-libreria-9",
      "metodoServicio": "ConsultarAtributosTransaccion03",
      "tipoTransaccion": "201000101"
    },
    "detalleMicroServicio": {
      "nombreServicio": "WSClientes0001",
      "metodoServicio": "ConsultarCuentasActivas01",
      "tipoTransaccion": "201000101",
      "fechaHoraInicio": "202603191501422530",
      "fechaHoraFin": "202603191501426410",
      "lotElastico": {
        "documento": {
          "fechaContable": "",
          "numeroDocumento": "",
          "journals": {
            "journal": {
              "valor": ""
            }
          }
        },
        "bodyIn": {
          "identificacion": "1111111111",
          "tipoIdentificacion": "0001",
          "tipoCuenta": "T"
        },
        "bodyOut": {
          "ahorros": {
            "ahorro": [
              {
                "numeroCuenta": "2200900207",
                "tipo": "TRANSACCIONAL TRADICIONAL"
              },
              {
                "numeroCuenta": "2203993563",
                "tipo": "Cuenta de AhorroTransaccional"
              }
            ]
          }
        }
      },
      "error": {
        "codigo": 0,
        "mensaje": "TCSBRKR3_BP-OK",
        "mensajeNegocio": null,
        "tipo": "INFO"
      }
    }
```

```
  }
}
```